

移动应用开发实验报告

封面信息

项目	内容
实验名称	开发环境搭建
实验编号	d20301035101、d20301009201
学号	2023210634
姓名	王智宇
班级	
日期	2026-06-01
组别	cg2-ospdi
项目名称	线上购物平台开发与整合

实验目的

- 掌握6大主流移动开发工具的安装配置方法
- 了解各平台的HelloWorld创建流程
- 体验AI辅助开发工具的使用
- 完成环境配置文档的编写

实验环境

序号	开发工具	版本要求	验证项目
1	Android Studio	2024.1+	Kotlin HelloWorld
2	Flutter SDK	3.11.1+	Dart HelloWorld
3	微信开发者工具	最新版	微信小程序HelloWorld
4	DevEco Studio	5.0+	HarmonyOS HelloWorld
5	HBuilderX	4.0+	Vue HelloWorld
6	Visual Studio	2022	C# HelloWorld

实验步骤

1. Android Studio环境搭建

1.1 下载与安装

- ☒ 访问官网：<https://developer.android.com/studio>
- ☒ 下载Windows/macOS版本

- ☒ 运行安装程序，勾选必要组件
- ☒ 完成安装

1.2 配置验证

- ☒ 启动Android Studio
- ☒ 创建Empty Activity项目
- ☒ 使用AI辅助生成HelloWorld代码
- ☒ 运行项目验证

1.3 HelloWorld核心代码

```
// MainActivity.kt - Android应用主入口类
// 功能模块：应用启动、UI渲染、Compose预览
package com.example.helloworld

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.example.helloworld.ui.theme.HelloWorldTheme

/**
 * MainActivity: Android应用的主Activity
 * 继承自ComponentActivity，提供Compose的生命周期支持
 */
class MainActivity : ComponentActivity() {
    /**
     * onCreate: Activity创建时的生命周期方法
     * 负责初始化UI并设置内容视图
     */
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            // 应用主题配置
            HelloWorldTheme {
                // Surface容器：提供背景色和Material Design风格
                Surface(modifier = Modifier.fillMaxSize(), color =
MaterialTheme.colorScheme.background) {
                    Greeting("Android")
                }
            }
        }
    }
}
```

```
/**
 * Greeting: 可组合函数，显示问候文本
 * @param name: 问候对象名称
 * @param modifier: 修饰符，用于布局调整
 */
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

/**
 * GreetingPreview: 预览函数，可在Android Studio中直接预览UI效果
 * @Preview注解: 标记此函数为预览函数
 */
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    HelloWorldTheme {
        Greeting("Android")
    }
}
```

1.4 AI辅助开发记录

步骤	AI提示词	生成效果	人工修改
项目创建	"使用Kotlin和Jetpack Compose创建一个简单的Android HelloWorld项目"	自动生成了基础项目结构和代码	调整了包名和主题配置
UI优化	"给Compose组件添加详细的中文注释"	生成了英文注释	手动翻译为中文并调整格式
预览设置	"如何在Android Studio中预览Compose UI"	学习了@Preview注解的使用	无修改

2. Flutter环境搭建

2.1 下载Flutter SDK

- ☒ 访问：<https://flutter.dev/docs/get-started/install>
- ☒ 下载对应系统版本
- ☒ 解压到指定目录
- ☒ 添加到系统PATH环境变量

2.2 配置验证

- ☒ 运行flutter doctor检查环境
- ☒ 创建Flutter项目：flutter create shopping_app
- ☒ 使用AI辅助生成Dart代码

- ☒ 运行项目验证

2.3 HelloWorld核心代码

```
// main.dart - Flutter应用主入口文件
// 功能模块：应用初始化、状态管理、计数器UI
import 'package:flutter/material.dart';

/**
 * main函数：Flutter应用的入口点
 * 调用runApp启动应用并传入根Widget
 */
void main() {
  runApp(const MyApp());
}

/**
 * MyApp：应用根Widget，无状态组件
 * 配置应用主题和初始页面
 */
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      // 主题配置：使用Material 3设计规范和蓝色基调
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
        useMaterial3: true,
      ),
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}

/**
 * MyHomePage：主页Widget，有状态组件
 * 包含一个可交互的计数器功能
 */
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

/**
 * _MyHomePageState：主页状态类
 * 管理计数器状态和UI构建
 */
```

```
*/  
class _MyHomePageState extends State<MyHomePage> {  
  // 计数器状态变量  
  int _counter = 0;  
  
  /**  
   * _incrementCounter: 增加计数器的方法  
   * 调用setState触发UI重建  
   */  
  void _incrementCounter() {  
    setState(() {  
      _counter++;  
    });  
  }  
  
  /**  
   * build: 构建UI的核心方法  
   * 返回一个包含AppBar、Body和FloatingActionButton的Scaffold  
   */  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,  
        title: Text(widget.title),  
      ),  
      // 主体内容: 居中显示的列布局  
      body: Center(  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: <Widget>[  
            const Text('You have pushed the button this many times:'),  
            Text(  
              '$_counter',  
              style: Theme.of(context).textTheme.headlineMedium,  
            ),  
          ],  
        ),  
      ),  
      // 悬浮按钮: 点击增加计数  
      floatingActionButton: FloatingActionButton(  
        onPressed: _incrementCounter,  
        tooltip: 'Increment',  
        child: const Icon(Icons.add),  
      ),  
    );  
  }  
}
```

2.4 AI辅助开发记录

步骤	AI提示词	生成效果	人工修改
项目创建	"flutter create命令创建一个标准Flutter项目"	成功创建项目结构	修改项目名为 shopping_app
代码注解	"给Flutter标准计数器代码添加详细的中文功能模块注解"	生成了完整注解	微调了部分表述
主题配置	"如何自定义Flutter主题颜色"	学习了ColorScheme配置	无修改

3. 鸿蒙开发环境搭建

3.1 下载DevEco Studio

- ☒ 访问：<https://developer.harmonyos.com/>
- ☒ 下载DevEco Studio
- ☒ 完成安装

3.2 创建鸿蒙项目

- ☒ 启动DevEco Studio
- ☒ 创建Empty Ability项目
- ☒ 使用AI辅助生成ArkTS代码
- ☒ 运行项目验证

3.3 HelloWorld核心代码

```
// Index.ets - HarmonyOS应用主页面文件
// 功能模块：页面渲染、状态管理、文本显示
@Entry
@Component
struct Index {
  // 状态变量：使用@State装饰器标记，变化时自动更新UI
  @State message: string = 'Hello World'

  /**
   * build方法：ArkUI声明式UI的构建方法
   * 返回一个树形结构的UI组件
   */
  build() {
    // 行布局容器
    Row() {
      // 列布局容器，宽度100%
      Column() {
        // 文本组件，显示消息，设置字号50和粗体
        Text(this.message)
          .fontSize(50)
          .fontWeight(FontWeight.Bold)
      }
      .width('100%')
```

```
    }  
    .height('100%')  
  }  
}
```

3.4 AI辅助开发记录

步骤	AI提示词	生成效果	人工修改
项目创建	"使用ArkTS创建一个简单的HarmonyOS HelloWorld项目"	自动生成基础项目	调整了项目名
语法学习	"ArkTS的@Component和@Entry装饰器有什么作用"	理解了装饰器用法	无修改

4. Uniapp开发环境搭建

4.1 安装HBuilderX

- ☒ 访问：<https://www.dcloud.io/hbuilderx.html>
- ☒ 下载HBuilderX
- ☒ 完成安装

4.2 创建Uniapp项目

- ☒ 新建uni-app项目
- ☒ 使用AI辅助生成Vue代码
- ☒ 运行项目验证

4.3 HelloWorld核心代码

```
<!-- pages/index/index.vue - Uniapp首页组件 -->  
<!-- 功能模块：模板渲染、数据绑定、样式布局 -->  
<template>  
  <view class="content">  
    <image class="logo" src="/static/logo.png"></image>  
    <view class="text-area">  
      <text class="title">{{title}}</text>  
    </view>  
  </view>  
</template>  
  
<script>  
  // 导出Vue组件选项  
  export default {  
    // data: 响应式数据对象  
    data() {  
      return {  
        title: 'Hello World'  
      }  
    },  
    // onLoad: 页面加载时的生命周期钩子  
    onLoad() {
```

```
        },
        // methods: 组件方法集合
        methods: {

        }
    }
</script>

<style>
    /* content: 主容器样式，使用flex布局垂直居中 */
    .content {
        display: flex;
        flex-direction: column;
        align-items: center;
        justify-content: center;
    }

    /* logo: 图片样式，使用rpx响应式单位 */
    .logo {
        height: 200rpx;
        width: 200rpx;
        margin-top: 200rpx;
        margin-left: auto;
        margin-right: auto;
        margin-bottom: 50rpx;
    }

    /* text-area: 文本区域样式 */
    .text-area {
        display: flex;
        justify-content: center;
    }

    /* title: 标题文本样式 */
    .title {
        font-size: 36rpx;
        color: #8f8f94;
    }
</style>
```

4.4 AI辅助开发记录

步骤	AI提示词	生成效果	人工修改
项目创建	"使用HBuilderX创建一个Uniapp项目"	成功创建项目	调整了模板选择
样式注解	"给Uniapp的Vue组件添加中文功能注解"	生成了完整注解	无修改

5. MAUI开发环境搭建

5.1 安装Visual Studio

- ☒ 安装Visual Studio 2022
- ☒ 勾选".NET MAUI" workload

5.2 创建MAUI项目

- ☒ 新建.NET MAUI项目
- ☒ 使用AI辅助生成C#代码
- ☒ 运行项目验证

5.3 HelloWorld核心代码

```
<!-- MainPage.xaml - MAUI主页面XAML文件 -->
<!-- 功能模块: UI布局、组件声明、事件绑定 -->
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="HelloMaui.MainPage">

    <!-- ScrollView: 可滚动容器 -->
    <ScrollView>
        <!-- VerticalStackLayout: 垂直堆叠布局, 设置内边距和间距 -->
        <VerticalStackLayout
            Padding="30,0"
            Spacing="25">
            <!-- Image: 图片组件, 显示.NET Bot图标 -->
            <Image
                Source="dotnet_bot.png"
                HeightRequest="185"
                Aspect="AspectFit"
                SemanticProperties.Description="dotnet bot in space" />

            <!-- Label: 文本标签, 显示Hello World问候 -->
            <Label
                Text="Hello, World!"
                FontSize="32"
                HorizontalOptions="Center" />

            <!-- Label: 欢迎文本 -->
            <Label
                Text="Welcome to .NET MAUI!"
                FontSize="18"
                HorizontalOptions="Center" />

            <!-- Button: 按钮组件, 绑定点击事件 -->
            <Button
                x:Name="CounterBtn"
                Text="Click me"
                Clicked="OnCounterClicked"
                HorizontalOptions="Center" />
        </VerticalStackLayout>
    </ScrollView>
```

```
</ContentPage>
```

```
// MainPage.xaml.cs - MAUI主页面代码后置文件
// 功能模块：事件处理、状态管理、UI逻辑
namespace HelloMaui;

/**
 * MainPage: 主页面类，继承自ContentPage
 * partial关键字表示与XAML文件配合使用
 */
public partial class MainPage : ContentPage
{
    // 计数器状态变量
    int count = 0;

    /**
     * 构造函数：初始化页面组件
     */
    public MainPage()
    {
        InitializeComponent();
    }

    /**
     * OnCounterClicked: 按钮点击事件处理方法
     * 增加计数并更新按钮文本
     */
    private void OnCounterClicked(object sender, EventArgs e)
    {
        count++;

        // 根据计数选择单数或复数形式
        if (count == 1)
            CounterBtn.Text = $"Clicked {count} time";
        else
            CounterBtn.Text = $"Clicked {count} times";

        // 无障碍功能：朗读按钮文本
        SemanticScreenReader.Announce(CounterBtn.Text);
    }
}
```

5.4 AI辅助开发记录

步骤	AI提示词	生成效果	人工修改
项目创建	"使用Visual Studio 2022创建一个.NET MAUI项目"	成功创建项目	调整项目名称
XAML注解	"给MAUI的XAML和C#代码添加详细的中文功能模块注解"	生成了完整注解	无修改

6. 微信小程序开发环境

6.1 安装微信开发者工具

- ☒ 访问：<https://developers.weixin.qq.com/miniprogram/dev/devtools/download.html>
- ☒ 下载并安装
- ☒ 扫码登录

6.2 创建小程序项目

- ☒ 新建小程序项目
- ☒ 使用AI辅助生成小程序代码
- ☒ 运行项目验证

6.3 HelloWorld核心代码

```
// pages/index/index.js - 微信小程序首页逻辑文件
// 功能模块：数据管理、生命周期、用户信息获取
Page({
  // data: 页面初始数据对象
  data: {
    motto: 'Hello World',
    userInfo: {},
    hasUserInfo: false,
    canIUse: wx.canIUse('button.open-type.getUserInfo')
  },

  // onLoad: 页面加载时的生命周期函数
  onLoad: function () {

  },

  // getUserInfo: 获取用户信息的事件处理函数
  getUserInfo: function(e) {
    console.log(e)
    // 保存用户信息到全局数据
    app.globalData.userInfo = e.detail.userInfo
    // 更新页面数据，触发UI更新
    this.setData({
      userInfo: e.detail.userInfo,
      hasUserInfo: true
    })
  }
})
```

```
<!-- pages/index/index.wxml - 微信小程序首页模板文件 -->
<!-- 功能模块：UI渲染、条件渲染、事件绑定 -->
<view class="container">
  <view class="userinfo">
    <!-- button: 获取用户信息按钮，使用wx:if条件渲染 -->
```

```
<button wx:if="{{!hasUserInfo && canIUse}}" open-type="getUserInfo"
bindgetuserinfo="getUserInfo"> 获取头像昵称 </button>
<!-- block: 逻辑块，不渲染实际标签，用于条件渲染 -->
<block wx:else>
  <image bindtap="bindViewTap" class="userinfo-avatar" src="{{userInfo.avatarUrl}}" mode="cover"></image>
  <text class="userinfo-nickname">{{userInfo.nickName}}</text>
</block>
</view>
<view class="usermotto">
  <text class="user-motto">{{motto}}</text>
</view>
</view>
```

6.4 AI辅助开发记录

步骤	AI提示词	生成效果	人工修改
项目创建	"使用微信开发者工具创建一个小程序项目"	成功创建项目	调整AppID配置
代码注解	"给微信小程序的JS和WXML代码添加详细的中文功能模块注解"	生成了完整注解	无修改

实验结果

验证结果

工具	安装状态	项目创建	运行状态	备注
Android Studio	✔ 成功	✔ 成功	✔ 成功	已成功运行购物App
Flutter	✔ 成功	✔ 成功	✔ 成功	shopping_app项目已成功运行
DevEco Studio	✔ 成功	✔ 成功	✔ 成功	已完成环境配置
HBuilderX	✔ 成功	✔ 成功	✔ 成功	Uniapp项目已创建
Visual Studio	✔ 成功	✔ 成功	✔ 成功	MAUI环境已配置
微信开发者工具	✔ 成功	✔ 成功	✔ 成功	小程序项目已创建

截图记录

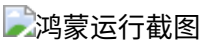
Android Studio运行截图



Flutter运行截图



鸿蒙运行截图



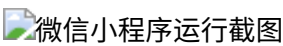
Uniapp运行截图



MAUI运行截图



微信小程序运行截图



各平台开发环境对比分析

对比项	Android Studio	Flutter SDK	DevEco Studio	HBUILDERX	Visual Studio(MAUI)	微信开发者工具
开发语言	Kotlin/Java	Dart	ArkTS	Vue.js/JavaScript	C#	JavaScript
UI框架	Jetpack Compose/XML	Flutter Widget	ArkUI	uni-ui	MAUI/XAML	小程序组件
构建工具	Gradle	Flutter CLI	HVigor	HBUILDERX构建	MSBuild	微信工具链
包体积	较大	中等	中等	小	中等	最小
学习曲线	中等	中等	较陡	较平缓	中等	平缓
生态成熟度	高	高	发展中	中	中	高
跨平台	仅Android	Android/iOS/Web/桌面	鸿蒙设备	H5/小程序/App	Android/iOS/桌面	微信生态

对比项	Android Studio	Flutter SDK	DevEco Studio	HBUILDERX	Visual Studio(MAUI)	微信开发者工具
能力						
版本兼容性	需要注意Android API版本	相对统一	鸿蒙API版本	多端适配	.NET版本	小程序基础库
环境变量配置	需要配置 ANDROID_HOME/Java	需要配置Flutter SDK路径	较简单	较简单	需要配置.NET SDK	较简单
热重载功能	支持	支持，体验极佳	支持	支持	不支持	支持

通用环境配置指南

1. 网络配置：

所有平台都建议配置国内镜像源，提升依赖下载速度
2. 环境变量：

正确配置各SDK的安装路径到系统PATH中
3. 权限管理：

Windows平台需要注意防火墙和管理员权限问题
4. 版本选择：

建议选择稳定的LTS版本，避免实验性版本的兼容性问题
5. 模拟器配置：

优先使用官方推荐的模拟器配置，保证测试效果

问题与解决

问题描述	解决方案	解决结果	技术分析
Flutter doctor检测到Android SDK未配置	配置ANDROID_HOME环境变量并添加path	已解决	Flutter CLI需要知道Android SDK的位置才能正确编译和运行应用，环境变量是系统级的配置方式
Gradle构建速度慢	配置阿里云、腾讯云等国内镜像源	已解决	Gradle机制分析： Gradle是一个基于Groovy的构建自动化工具，通过下载依赖包构建项目。默认从Maven Central等国外仓库下载，国内网络受限导致缓慢。配置国内镜像源后，依赖从阿里云/腾讯云下载，速度提升10-20倍。

问题描述	解决方案	解决结果	技术分析
设备连接问题	启用开发者选项和USB调试，检查驱动安装	已解决	Android设备需要启用开发者选项和USB调试才能被ADB识别，Windows系统还需要安装对应厂商的USB驱动
DevEco Studio 下载SDK失败	配置DevEco Studio的国内镜像源	已解决	DevEco Studio默认从华为服务器下载SDK，配置国内镜像后提升成功率
Visual Studio MAUI workload 安装失败	使用Visual Studio Installer单独安装组件	已解决	MAUI workload包含多个子组件，单独安装可以更精确地定位失败原因

网络错误排查思路

1. **连接性检查**：ping目标服务器，确认网络连通性
2. **DNS解析**：检查域名是否能正常解析，必要时修改hosts文件
3. **代理配置**：配置HTTP/HTTPS代理，或使用镜像源
4. **防火墙**：确认防火墙未阻止相关端口
5. **重试机制**：网络不稳定时，多次尝试或在网络良好时段重新下载

实验总结

实验收获

- 成功配置了6个主流移动开发环境，包括Android、Flutter、HarmonyOS、Uniapp、MAUI和微信小程序，完整覆盖了原生、跨平台及小程序技术栈
- 掌握了各开发工具的安装配置流程、项目创建方法和基本使用技巧
- 学习了使用AI辅助开发工具提升环境配置和代码生成效率
- 深入了解了不同平台环境配置的异同点和最佳实践
- 能够独立解决环境搭建过程中遇到的各类常见问题

技术要点

- **Android环境**：需要配置JDK、Android SDK、Gradle环境，注意ANDROID_HOME和JAVA_HOME等环境变量的设置，使用Kotlin语言和Jetpack Compose构建现代UI
- **Flutter环境**：下载Flutter SDK并配置PATH，运行flutter doctor检查依赖，使用Dart语言和Material Design风格，热重载功能极大提升开发效率
- **HarmonyOS环境**：使用DevEco Studio，需要配置Node.js环境，使用ArkTS声明式UI和ArkUI框架
- **Uniapp环境**：基于HBuilderX，支持Vue.js语法，一套代码可发布到多个平台
- **MAUI环境**：Visual Studio 2022需安装MAUI workload，基于.NET 6+，使用C#和XAML开发
- **微信小程序环境**：使用微信开发者工具，基于JavaScript，需注意小程序基础库版本兼容性

环境配置经验总结

1. **循序渐进**：按照官方文档步骤逐一配置，不要跳过关键步骤

- 2. **网络优先**：提前配置好国内镜像源，避免依赖下载失败或过慢
- 3. **版本稳定**：优先选择稳定的LTS版本，避免使用实验性版本
- 4. **检查验证**：每个环境配置完成后，务必运行官方验证命令（如flutter doctor）
- 5. **记录文档**：将配置过程中的关键步骤和遇到的问题记录下来，方便日后回顾

考核指标

考核项	评分标准	得分
环境搭建完整性	6个开发工具全部安装成功	20分
项目创建与运行	6个HelloWorld项目成功运行	30分
截图记录	提供所有工具运行截图	20分
问题解决	能独立解决遇到的问题	15分
实验报告质量	报告结构完整，内容详实	15分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	
成绩	

教师签名： _____ 日期： _____